

UNIVERZA V LJUBLJANI
FAKULTETA ZA MATEMATIKO IN FIZIKO
Oddelek za fiziko

6. naloga: Gručenje v hadronske curke

Praktikum strojnega učenja v fiziki,
Fizika, 2. stopnja, 2. letnik

Avtor: Kevin Jaksetič, vpisna številka: 28222050
Predavatelj pri nalogi: prof. dr. Jernej Fesl Kamenik
Asistent pri nalogi: Blaž Bortolato

Študijsko leto 2023/2024

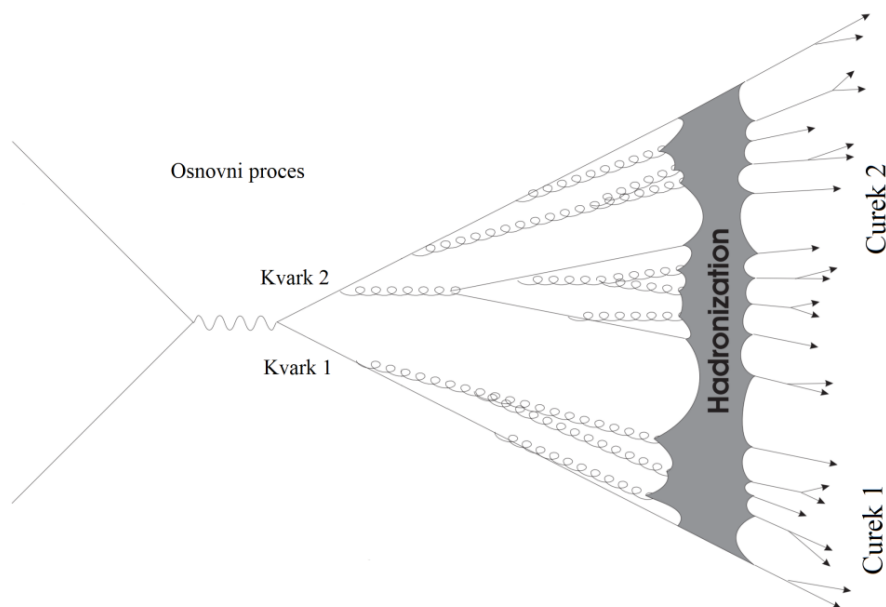
Kazalo

1	Teoretični uvod: hadronski curki	2
1.1	Algoritmi gručenja	3
1.1.1	Algoritem K-means	3
1.2	Hierarhično gručenje	4
2	Navodila naloge	5
3	Reševanje z algoritmom K-means	6
3.1	Preizkus na testnih podatkih	6
3.2	Gručenje pravih podatkov	7
3.3	Masa Higgsovega bozona	14
3.4	Testiranje infrardeče varnosti	15
4	Hierarhično gručenje	18
4.1	Izračun mase Higgsovega bozona iz vseh dogodkov	22
4.2	Vizualna reprezentacija gruč	23

1 Teoretični uvod: hadronski curki

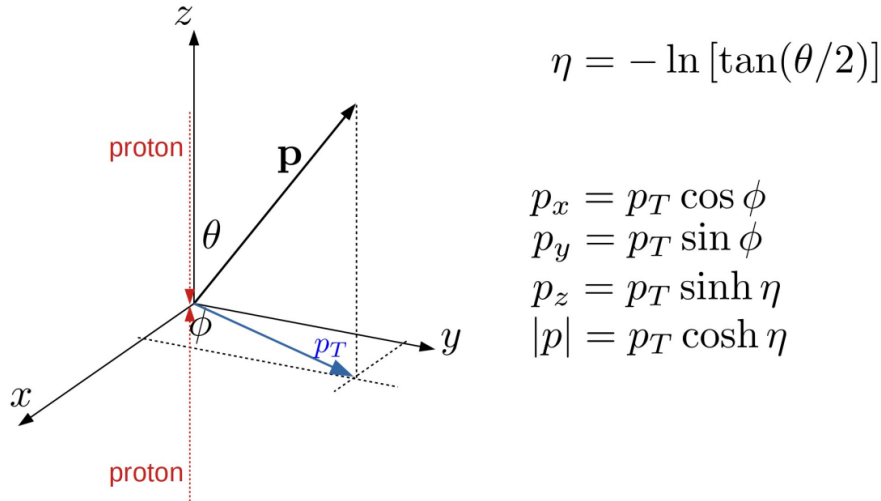
Kot posledico visoko-energijskih trkov med osnovnimi delci dostikrat pričakujemo veliko število **hadronov**. Ti so sestavljeni iz kvarkov, ki jih veže močna interakcija. Ker Standardni Model opisuje spekter osnovnih delcev in dinamiko med njimi, ne pa tudi eksplicitno vezanih stanj, se pojavi pomembno vprašanje: kako v procesih fizike osnovnih delcev (npr. v sipanju) povezati kvarkovska končna stanja s hadroni, ki jih dejansko zaznamo v eksperimentu? Prostih kvarkov namreč ne moremo zaznati, saj se zaradi narave močne interakcije hitro vežejo v hadrone. Poleg tega lahko močna interakcija povzroči tudi sevanje gluonov (prenašalcev močne interakcije) - iz kvarkov, ti pa nato spet tvorijo kvarke, ki lahko ponovno sevajo gluone, ali pa se vežejo v hadrone. To se lahko ponavlja, tako da v končni fazi v eksperimentu dobimo veliko število hadronov (**hadronizacija**), ki pa vsi izvirajo iz nekaj kvarkov iz osnovne interakcije (slika 1). Ti hadroni so pogosto precej kolimirani, tako da lahko ločimo skupine hadronov, ki izvirajo iz enega kvarka iz osnovne interakcije od ostalih. Vsako takšno skupino hadronov imenujemo **hadronski curek** - (angl. *hadronic jet*). Na našo veliko srečo transformacija od kvarkov do končnih hadronskih curkov ne spremeni signifikantno informacije o energiji in gibalni količini, zato lahko curke uporabljamo za študije procesov v fiziki osnovnih delcev in primerjavo s teorijo, kljub temu, da le-ta govori v jeziku kvarkov.

Če v nekem procesu v končnem stanju pričakujemo npr. dva kvarka, na eksperimentu načeloma pričakujemo dva hadronska curka. V praksi pa bo hadronskih curkov več. Na hadronskih trkalnikih, kot je recimo LHC (ki trka protone), lahko namreč tudi začetni stanji sevata gluone, cela zgodba s hadronizacijo pa se lahko ponovi tudi za ta stanja, kar rodi veliko dodatnih hadronov v končnem stanju. Ponavadi to težavo rešimo tako, da curke filtriramo na podlagi kakšne kinematične lastnosti, odvisno od tega, kakšen osnoven proces smo pričakovali: obravnavamo recimo samo nekaj najbolj visoko-energijskih curkov.



Slika 1: Shematski prikaz poteka hadronizacije iz dveh kvarkov v končnem stanju osnovnega procesa.

Da bomo lahko študirali kinematične lastnosti curkov, vpeljimo koordinatni sistem (glej sliko 2). Poljuben delec z maso m v končnem stanju lahko opišemo s četvercem gibalne količine $p = (E, \mathbf{p})$, kjer je $E = \sqrt{m^2 + |\mathbf{p}|^2}$ njegova energija, $\mathbf{p}$ pa vektor gibalne količine. V kontekstu visoko-energijskih trkalnikov osnovnih delcev lahko pogosto mase delcev zanemarimo, ter namesto koordinat (p_x, p_y, p_z) raje uporabljamo koordinate (p_T, η, ϕ) . Tukaj je p_T transverzalna gibalna količina, η psevdorapidnost, ϕ pa azimutni kot.



Slika 2: Skica koordinatnega sistema, uporabljenega v fiziki trkalnikov osnovnih delcev in transformacija med koordinatnima sistemoma.

Hadronski curki so torej kolimirane skupine hadronov. Ostalo pa nam je še vprašanje, kako izbrati, kateri hadron spada v katero skupino?

1.1 Algoritmi gručenja

Ker želimo iz velikega števila hadronov ustvariti skupine kolimiranih curkov, se obrnemo na algoritme gručenja, ki jih uporabljamo v ravnini kotnih spremenljivk (η, ϕ) . Pričakujemo namreč, da se bodo hadroni, ki izvirajo iz istega osnovnega kvarka, v tej ravnini nahajali bližje skupaj.

1.1.1 Algoritem K-means

Algoritem K-means je eden najenostavnejših algoritmov za gručenje, njegovo ime pa se nanaša na njegov osnovni princip delovanja. Izberemo si, da algoritem razdeli podatke na K gruč, gruče pa so opisane z njihovimi srednjimi vrednostmi, t.i. centri. Za začetne centre si lahko izberemo kar naključne točke iz podatkov. Nato ponavljamo dva koraka:

1. Na podlagi oddaljenosti vzorcev od trenutno določenih centroidov pripišemo vsakemu vzorcu pripadajočo gručo, tako da je razdalja med njim in pripadajočim centroidom minimalna.
2. Izračunamo povprečne vrednosti vzorcev vsake gruče in te vzamemo za nove vrednosti centroidov.

Ta dva koraka ponavljamo dokler se centriodi še spreminjajo. Vsak vzorec po konvergeni pripada gruči z najbližjim centroidom.

Dobljene gruče imenujemo curki, p_T vsakega pa izračunamo kot vsoto p_T vseh konstituentov. Algoritem pa ima bistveno težavo. Zaradi fizikalnega ozadja procesa hadronizacije namreč želimo, da ima dve pomembni lastnosti:

1. Infrardeča varnost: curki niso senzitivni na dodajanje novih delcev s $p_T \rightarrow 0$.
2. Kolinearna varnost: curki niso senzitivni, če delec z gibalno količino p_i zamenjamo z dvema vzporednima (kolinearnima) delcema j in k , tako da je $p_i = p_j + p_k$.

V ta namen so v skupnosti fizike osnovnih delcev nastali specializirani algoritmi, ki jih bomo vpeljali zdaj.

1.2 Hierarhično gručenje

Spoznali bomo tri algoritme, ki pa se med seboj razlikujejo le po definiciji metrike na koordinatah delcev. Vse tri algoritme lahko opišemo hkrati z naslednjim postopkom, s tem da vpeljemo parameter p , s katerim bomo razlikovali med algoritmi. Algoritem začne s seznamom protocurkov, to so vsi delci v končnem stanju enega dogodka, ter s praznim seznamom pravih curkov, ki pa se ob poteku algoritma polni. Algoritem ima prost parameter R , ki je poljubno število reda 1. Nato algoritem deluje tako:

1. Za vsak protocurek definiramo $d_i = p_{T,i}^{2p}$ ter za vsak par protocurkov definiramo

$$d_{ij} = \min(p_{T,i}^{2p}, p_{T,j}^{2p}) [(\eta_i - \eta_j)^2 + (\phi_i - \phi_j)^2] / R^2$$

2. Najdemo $d_{\min}$, to je najmanjša vrednost med vsemi d_i in d_{ij} .
3. Če je $d_{\min}$ ena od razdalj d_{ij} , združimo protocurka i in j v nov protocurek k z

$$\begin{aligned} p_{T,k} &= p_{T,i} + p_{T,j} \\ \eta_k &= (p_{T,i}\eta_i + p_{T,j}\eta_j) / p_{T,k} \\ \phi_k &= (p_{T,i}\phi_i + p_{T,j}\phi_j) / p_{T,k} \end{aligned}$$

4. Če pa je $d_{\min}$ eden od d_i , protocurek i odstranimo iz seznama protocurkov in ga promoviramo v seznam pravih curkov.

Korake ponavljamo, dokler ne izpraznimo seznama protocurkov, s čimer se postopoma polni seznam pravih curkov. Z izbiro parametra $p \in \{-1, 0, 1\}$ reproduciramo znane algoritme anti- k_t , Cambridge-Aachen in k_t , v takšnem vrstnem redu.

2 Navodila naloge

1. Implementiraj algoritem K-means, tako da slediš prej opisanem postopku. Preizkusi ga na priloženih podatkih v datoteki `gauss.npy`. Podatki so v obliki niza velikosti 400×2 . Predstavljajo 400 delcev vsak z dvema parametroma, ki sta naključno vzorčena iz porazdelitve, ki jo sestavljata Gaussova porazdelitev s povprečjem $(-1.5, -0.5)$ in standardnim odklonom 0.3 ter še ena Gaussova porazdelitev s povprečjem $(1.5, 0.75)$ in odklonom 0.5.

2. V datoteki `h_bb_sorted.npy` se nahaja niz dogodkov. Vsak dogodek vsebuje različno število delcev. Parametri vsake delca so podani v obliki *Python* tuplov (p_T, η, ϕ, m) (masa m ne pride v poštev pri nobeni nalogi). En dogodek je v tem primeru trk dveh protonov, pri katerem nastane poleg drugih delcev tudi Higgsov bozon, ki razpade na kvarka b in $\bar{b}$. Vrednosti so simulirane z upoštevanjem efekta hadronizacije.

Uporabi implementiran algoritem K-means na prvem dogodku iz datoteke za $K = 2$ in $K = 10$. V obeh primerih predvidi, da curka z največjim p_T prihajata iz razpada Higgsovega bozona - torej določi četverca gibalnih količin obeh curkov p_1 in p_2 ter določi maso Higgsovega bozona kot

$$m_H = \sqrt{(p_1 + p_2)_\mu (p_1 + p_2)^\mu} = \sqrt{(E_1 + E_2)^2 - (\mathbf{p}_1 + \mathbf{p}_2)^2}, \quad (1)$$

Pri čemer upoštevaj približek $E_{1,2} \approx |p_{1,2}|$. Podatke naložimo z ukazom: `data = np.load('h_bb_sorted.npy', allow_pickle=True)`. Masa Higgsovega bozona je 125.11 ± 0.11 GeV.

3. Preizkusi infrardečo varnost implementiranega algoritma K-means, tako da iz prvega dogodka postopoma odstranjuješ delce z najmanjšim p_T . Nariši odvisnost rekonstruirane mase Higgsovega bozona od števila delcev v dogodku.
4. Implementiraj algoritem k_t , tako da slediš prej opisanem postopku. Z uporabo parametra $R = 0.6$ ($\sim$ idealna vrednost za te podatke) izvedi test infrardeče varnosti. Kaj se zgodi če povečamo ali zmanjšamo R ?
5. Reproduciraj test infrardeče varnosti še z uporabo algoritma k_t iz knjižnice `pyjet` in primerjaj rezultate s svojim algoritmom.
6. Primerjaj časovno zahtevnost svojega algoritma z algoritmom iz `pyjet`, tako da ponovno postopoma odstranjuješ delce iz prvega dogodka z najmanjšim p_T in v vsakem koraku meriš čas izračuna.
7. Poženi algoritem iz knjižnice `pyjet` na vseh dogodkih iz datoteke `h_bb_sorted.npy` in nariši spekter rekonstruiranih invariantnih mas Higgsovega bozona.

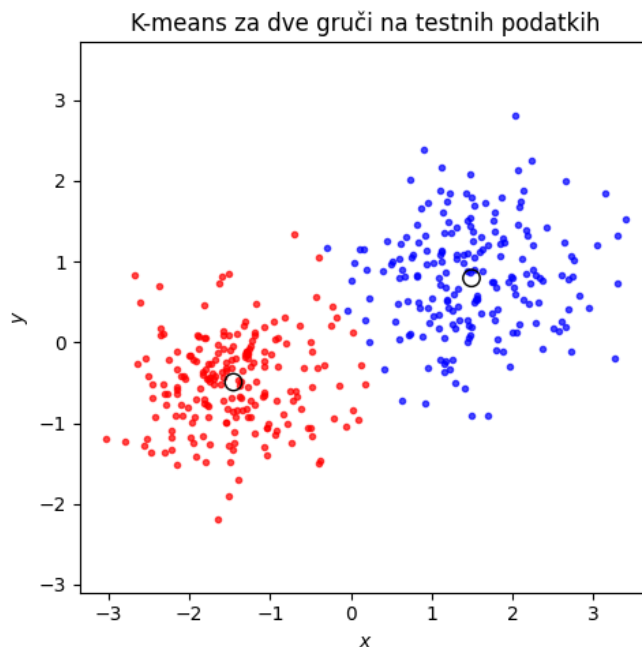
3 Reševanje z algoritmom K-means

V programskem jeziku *Python* sem spisal svoj algoritem gručenja K-means, ki za začetne vrednosti centroidov gruč vzame kar med seboj si različne naključne podatke iz seta. To je tudi razlog, da algoritem ni determinističen, vsakič namreč dobimo nekoliko drugačne rezultate. Zato je za resno študijo smiselno opazovati tako posamezne izračune kot tudi statistiko večjega števila računov.

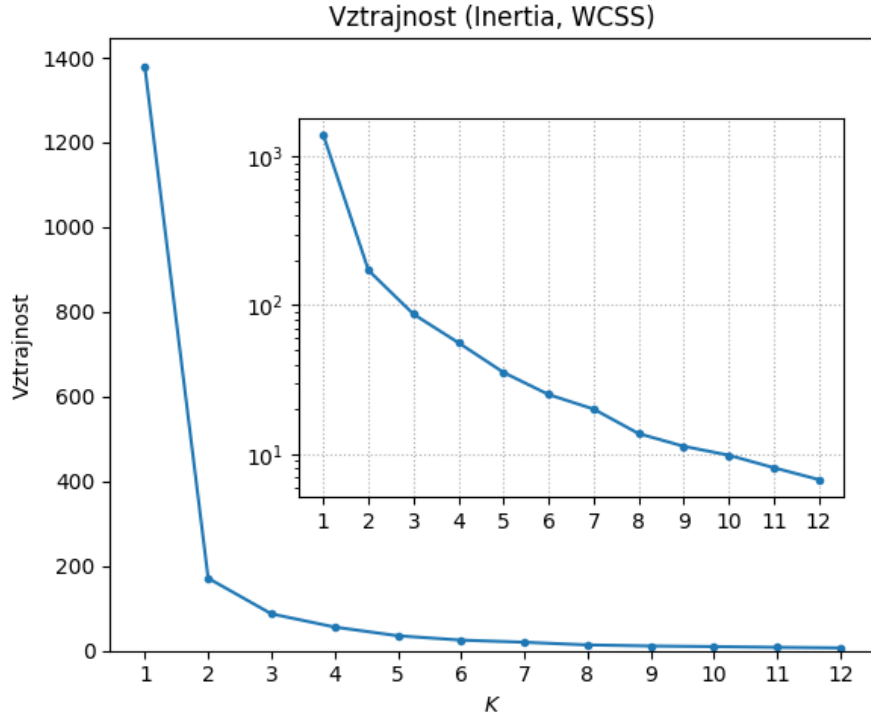
3.1 Preizkus na testnih podatkih

Algoritem sem najprej testiral na danem setu testnih podatkov iz datoteke `gauss.npy`, kjer so bili podatki izžrebani po dveh Gaussovih porazdelitvah. Od algoritma zahtevamo naj ustvari določeno število gruč. V primeru naših testnih podatkov sta optimalni in želeni dve gruči. Rezultati algoritma so podani na sliki 3.

Navadno pa ne vemo kolikšno število gruč je optimalno, zato se moramo izmisliti kakšnega testa, ki bi nam to povedal. Najpreprostejši izmed testov je tako imenovana **Metoda Komolca** (angl. *The Elbow Method*). Pri njej izračunamo povprečen kvadrat razdalje med točkami gruče in njenega centroida. To količino imenujemo **vztrajnost** (angl. *Inertia*). Rezultati metode komolca za naše testne podatke so prikazani na sliki 4, kjer lahko jasno vidimo „komolec“ pri številu gruč $K = 2$, ki nakazuje, da je to optimalno število gruč. Žal situacija ni vedno tako preprosta, dostikrat je padec vztrajnosti po številu gruč dosti bolj položen, zaradi česar ni mogoče gotovo izbrati optimalnega števila gruč (kot bomo videli kasneje na bolj realističnih podatkih). V takih primerih se je treba poslužiti kakšnega boljšega testa, kot je na primer **Metoda silhuete**.



Slika 3: Razdelitev testnih podatkov na dve gruči z uporabo algoritma K-means.

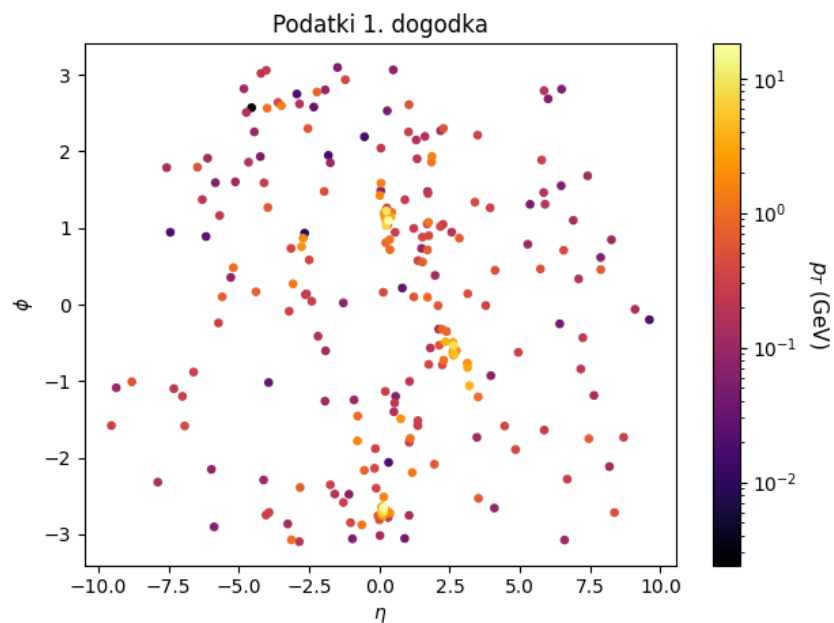


Slika 4: Metoda komolca (The Elbow Method) za algoritem K-means, za testne podatke sestavljene iz dveh Gaussovih porazdelitev. Pri številu gru $\check{c}$ $K = 2$ vidimo o $\check{c}$ itni „komo-
lec”, ki je torej indikacija optimalnega števila gru $\check{c}$ za na $\check{s}$ e testne podatke.

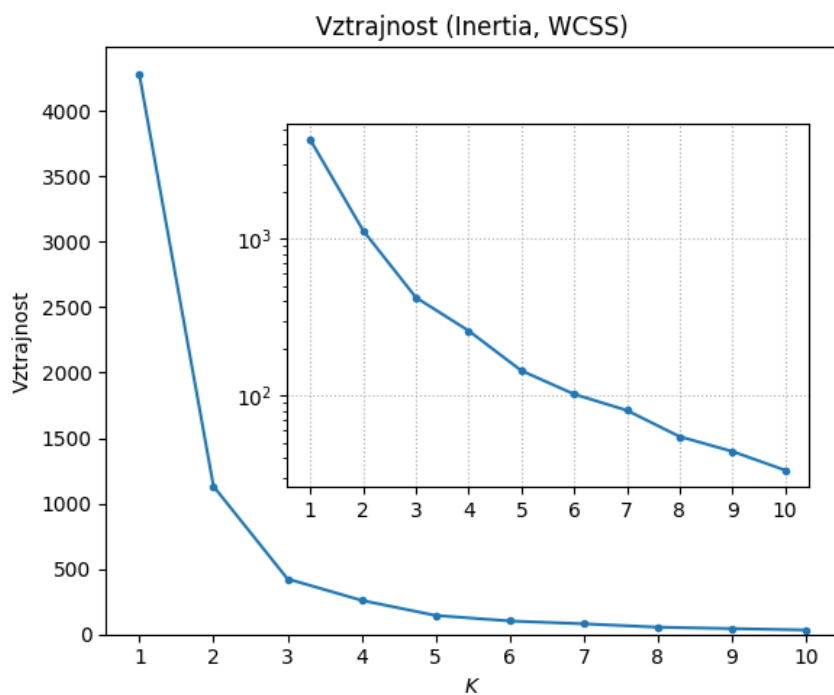
3.2 Gru $\check{c}$ enje pravih podatkov

V datoteki `h_bb_sorted.npy` imamo simulirane podatke za 10 000 dogodkov, kjer posamezen dogodek predstavlja trk dveh protonov, pri katerem poleg drugih delcev nastane tudi Higgsov bozon, ki razpade na kvarka b in $\bar{b}$. Podatki so bili simulirani z upoštevanjem efekta hadronizacije. Vsak dogodek $\check{z}$ elimo gru $\check{c}$ iti v ravnini (η, ϕ) iz slike 2, pri $\check{c}$ emer privzamemo, da gru $\check{c}$ i oz. curka z najve $\check{c}$ ijima p_T izvirata iz razpada Higgsovega bozona. Tako lahko dolo $\check{c}$ imo gibalni koli $\check{c}$ ini obeh curkov p_1 in p_2 , ter maso Higgsovega bozona po ena $\check{c}$ bi 1, kjer upo $\check{s}$ tevamo pribli $\check{z}$ ek $E_{1,2} \approx |p_{1,2}|$.

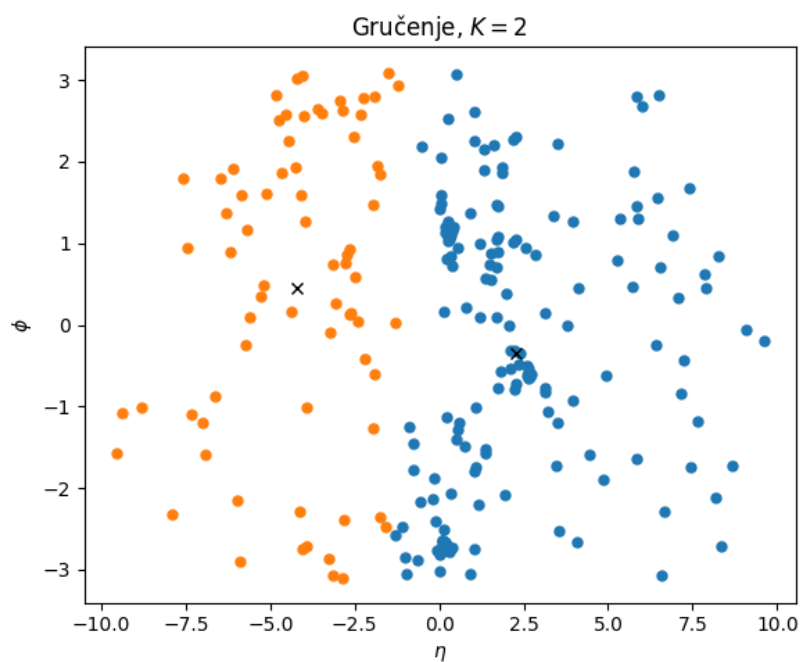
Za za $\check{c}$ etek si poglejmo rezultate gru $\check{c}$ enja na prvem dogodku iz seznama. Podatki so prikazani na sliki 5. Vidimo, da optimalno število gru $\check{c}$ ni ve $\check{c}$ o $\check{c}$ itno, opazimo pa lahko skupke delcev z veliko transversalno gibalno koli $\check{c}$ ino, ki gotovo spadajo v isto gru $\check{c}$ o. Na tej to $\check{c}$ ki preizkusimo gru $\check{c}$ enje z Metodo Komolca, ki smo jo opisali prej. Rezultati le-te so prikazani na sliki 6. Na podlagi testa pri $\check{c}$ akujemo, da je optimalno podatke razdeliti na dve do $\check{s}$ tiri gru $\check{c}$ e. Sedaj si lahko s prostim o $\check{c}$ esom pogledamo, na kak $\check{s}$ en na $\check{c}$ in si algoritem izbere gru $\check{c}$ e glede na njihovo število. Na slikah 7, 8, 9, 10 in 11 so prikazane re $\check{s}$ itve algoritma za število gru $\check{c}$ $K = \{2, 3, 4, 6, 10\}$. Na podlagi prikazov bi ocenil, da je optimalno število gru $\check{c}$ $K = 4$, v nadaljevanju pa bomo videli, da to ne pomeni nujno najbolj $\check{s}$ e napovedi za maso Higgsovega bozona.



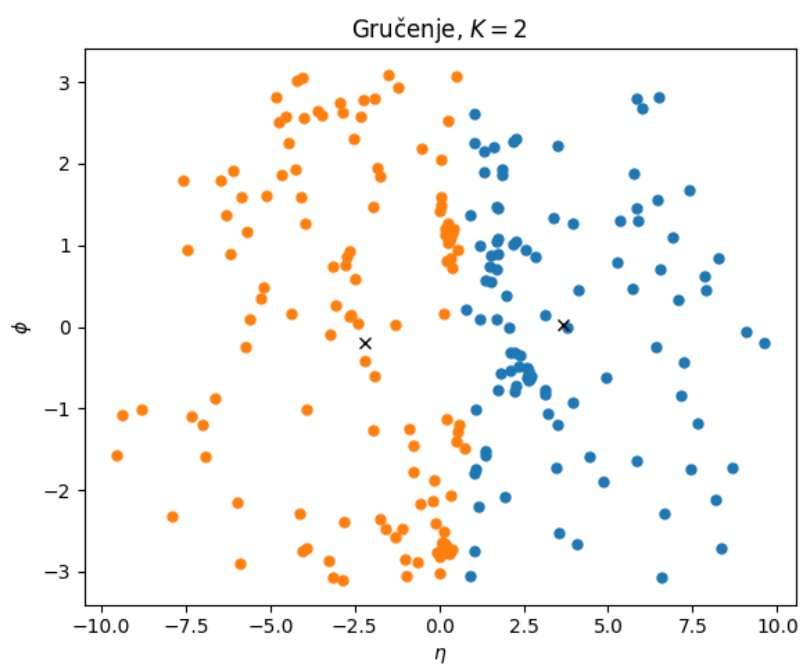
Slika 5: Podatki prvega dogodka iz seznama. Opazimo lahko skupke visokih p_T , optimalno število gruč pa ni očitno.



Slika 6: Metoda komolca za gručenje podatkov prvega dogodka iz seznama. Optimalno število gruč ni več tako očitno, kot je bilo na testnih podatkih.

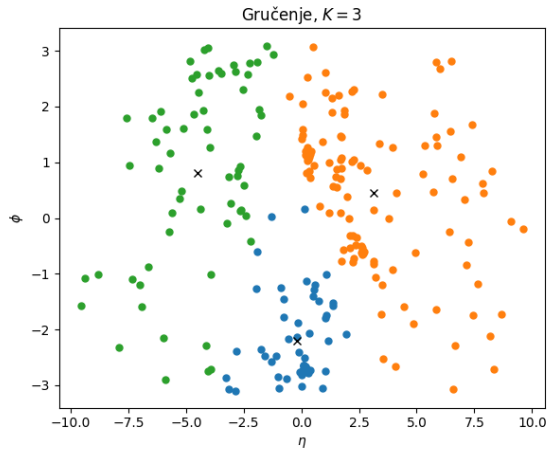


(a) Prvi poskus.

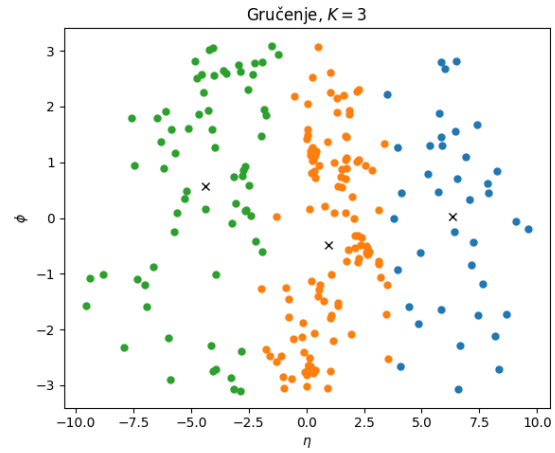


(b) Drugi poskus.

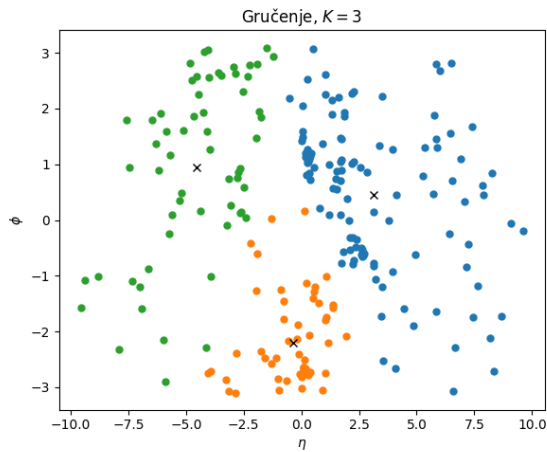
Slika 7: Razdelitev podatkov prvega dogodka na dve gruči z uporabo algoritma K-means. Opazil sem, da algoritem ne izbira gruči konsistentno, ob ponavljanju izračuna se pojavljata verziji iz prvega (levo) in drugega poskusa (desno). To gotovo ni znak za zadostno število gruči.



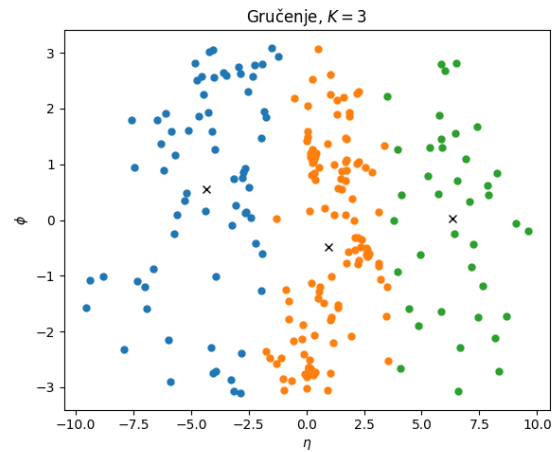
(a) Prvi poskus. Zaradi majhnega števila podatkov v sredini rešitev deluje v redu.



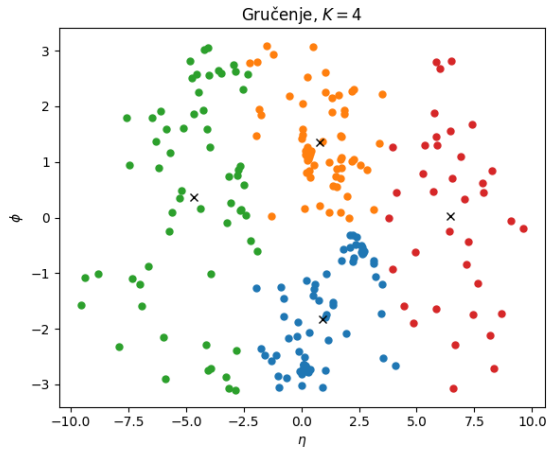
(b) Drugi in tretji poskus, ki sta skupke z veliko gibalno količino spravila v eno samo gručo (sredinsko).



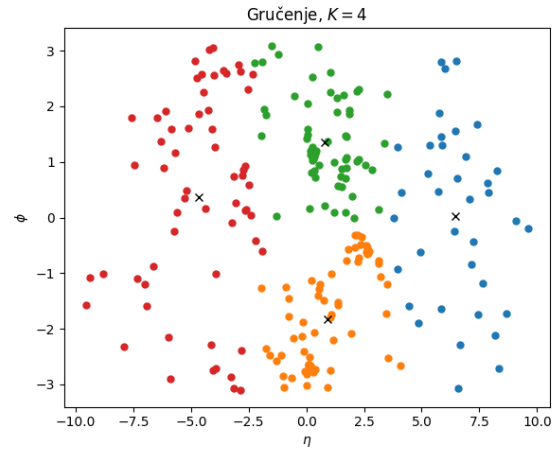
(c) Četrty poskus je nadvse podoben prvemu. (d) Peti poskus, ki se od drugega in tretjega razlikuje, ki se nahajajo levo spodaj.



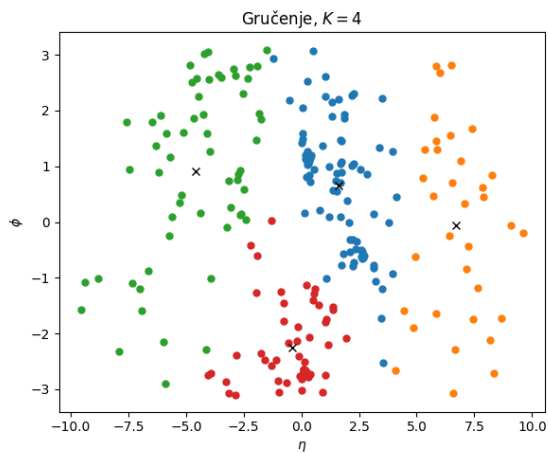
Slika 8: Razdelitev podatkov prvega dogodka na tri gručo z uporabo algoritma K-means. Kot rezultat dobimo nekaj različnih možnosti.



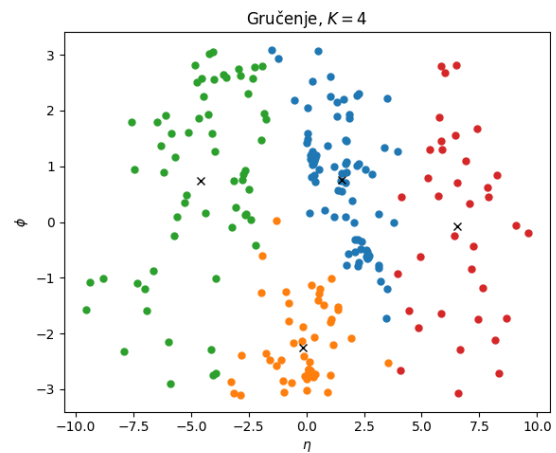
(a) Prvi poskus.



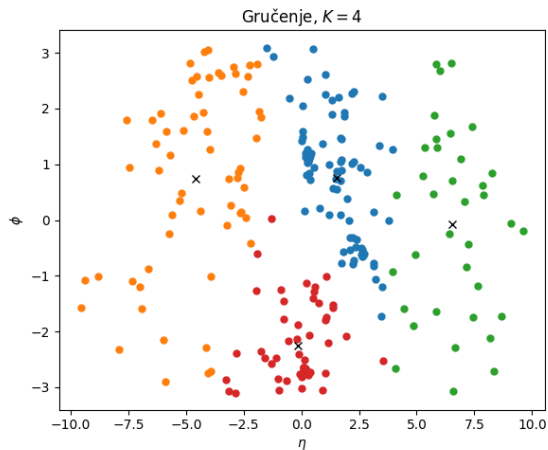
(b) Drugi poskus, ki je identičen prvemu.



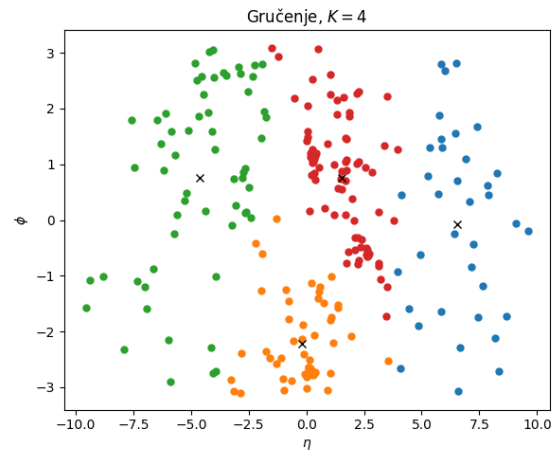
(c) Tretji poskus.



(d) Četrty poskus, zelo podoben tretjemu.

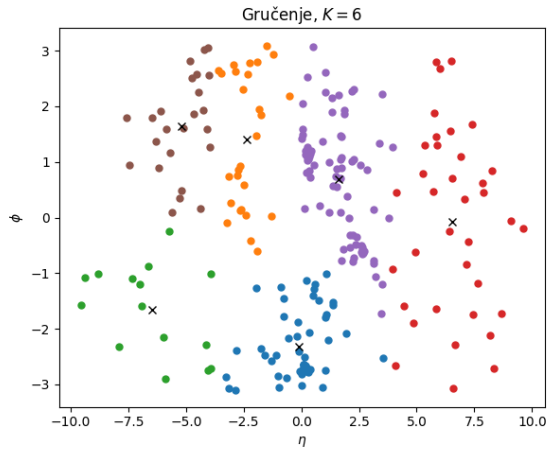


(e) Peti poskus, identičen četrtemu.

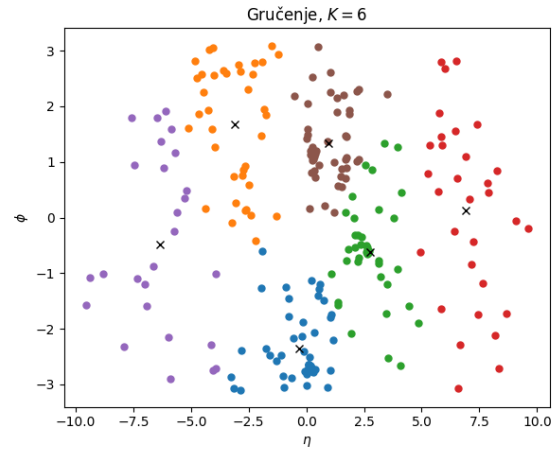


(f) Šesti poskus, nadvse podoben zadnjim trem.

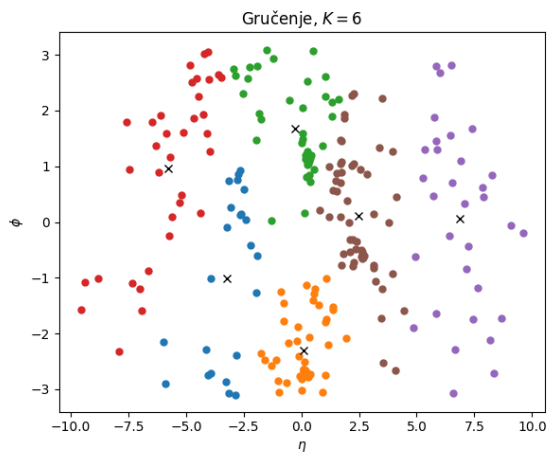
Slika 9: Razdelitev podatkov prvega dogodka na štiri gruče z uporabo algoritma K-means. Kot rezultat dobimo nekaj različnih možnosti. Različni poskusi so si med seboj precej podobni, kar je znak dobrega gručenja.



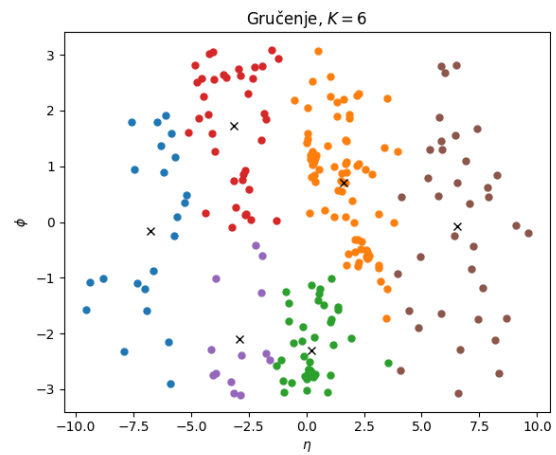
(a) Prvi poskus.



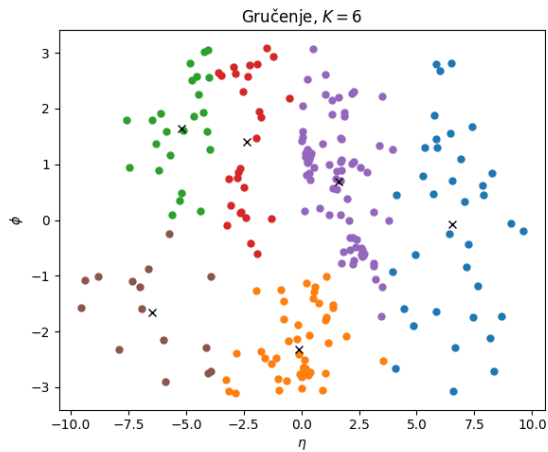
(b) Drugi poskus.



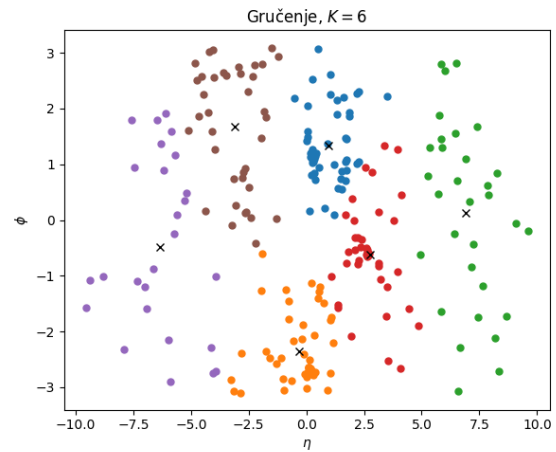
(c) Tretji poskus.



(d) Četrti poskus.

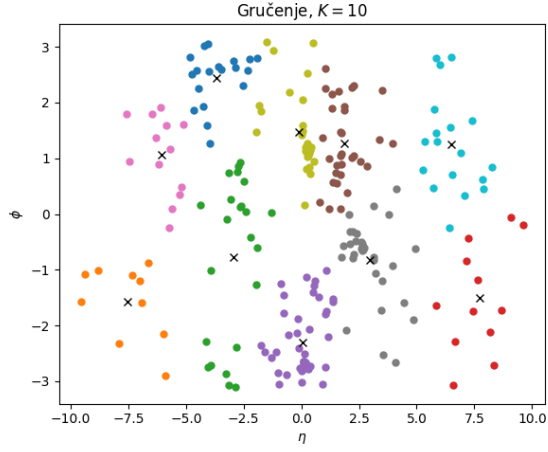


(e) Peti poskus.

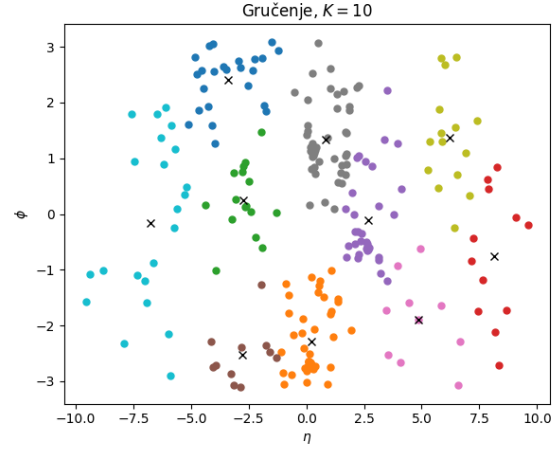


(f) Šesti poskus.

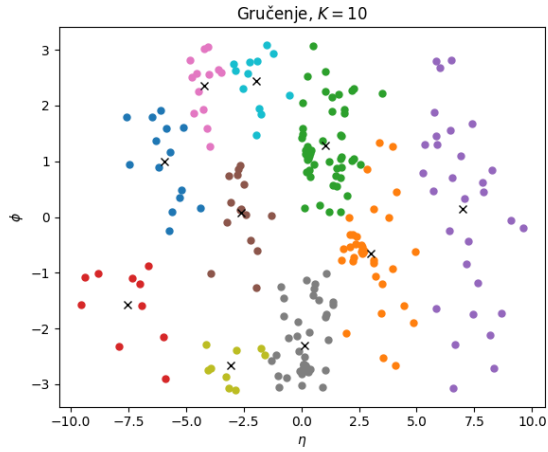
Slika 10: Razdelitev podatkov prvega dogodka na šest gruč z uporabo algoritma K-means. Izbira gruč je zelo nekonsistentna, kar je znak uporabe prevelikega števila gruč. Izgleda da sta desna ter spodnja sredinska gruča še najbolj robustni.



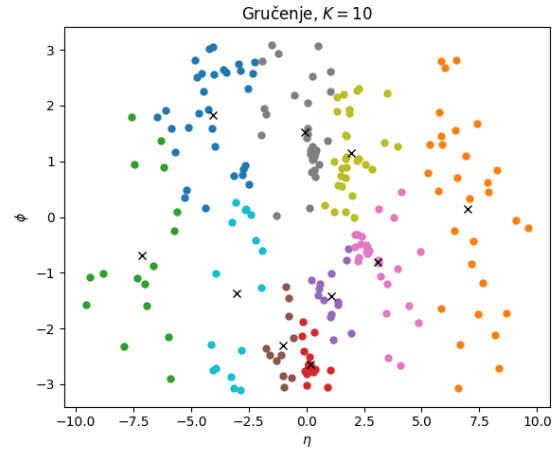
(a) Prvi poskus.



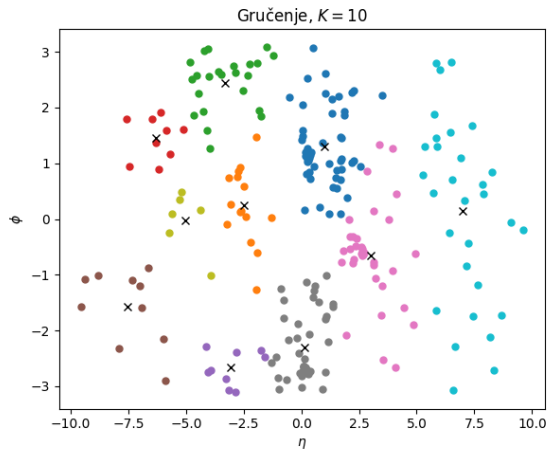
(b) Drugi poskus.



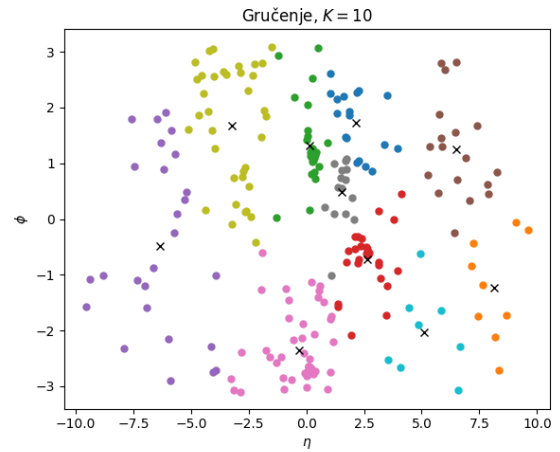
(c) Tretji poskus.



(d) Četrti poskus.



(e) Peti poskus.

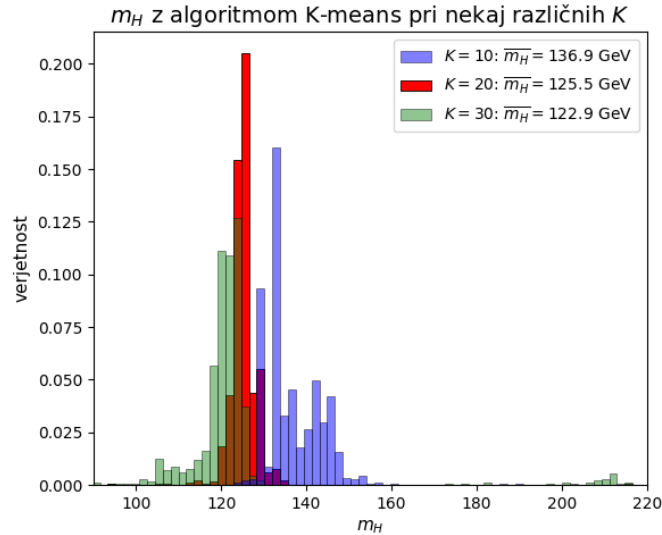


(f) Šesti poskus.

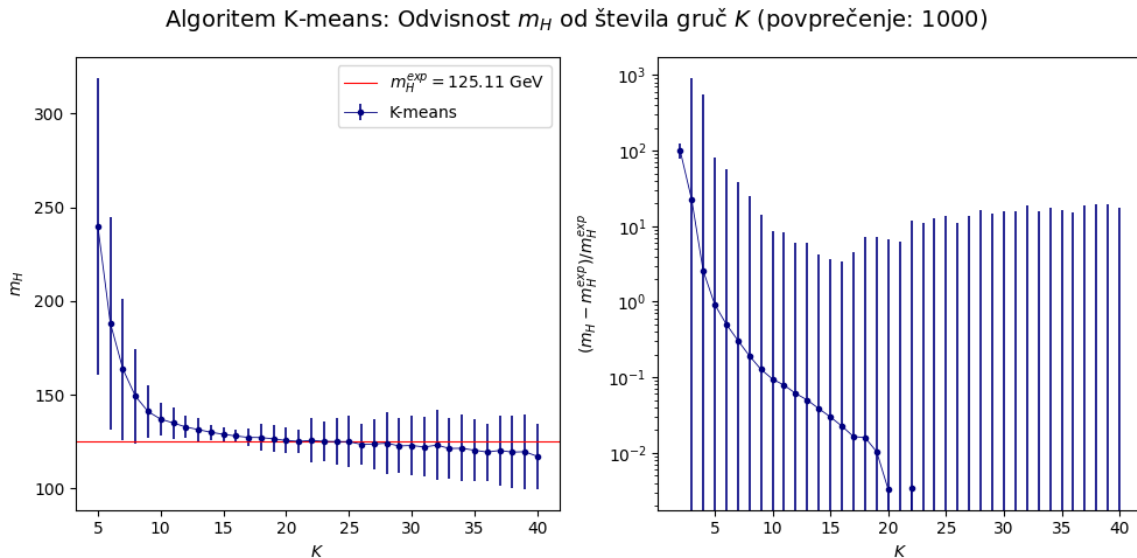
Slika 11: Razdelitev podatkov prvega dogodka na deset gruĉ z uporabo algoritma K-means. Izraĉun je oĉitno moĉno odvisen od zaĉetnih ugibanj za centroide, saj je gruĉenje zelo nekonsistentno.

3.3 Masa Higgsovega bozona

Napočil je čas, da analizo popeljemo v kvantitativno smer, in si pogledamo izračunane mase Higgsovega bozona po enačbi (1). Ker algoritem K-means ni determinističen, je smiselno preučiti tudi statistiko rešitev in ne le rezultatov enega poskusa. Rezultate bomo primerjali z eksperimentalno dobljeno maso $m_H^{\text{exp}} = 125.11 \pm 0.11$ GeV.



Slika 12: Histogrami m_H , izračunanih z algoritmom K-means za število gruč $\{10, 20, 30\}$. Za fiksno K sem algoritem pogljal tisočkrat.



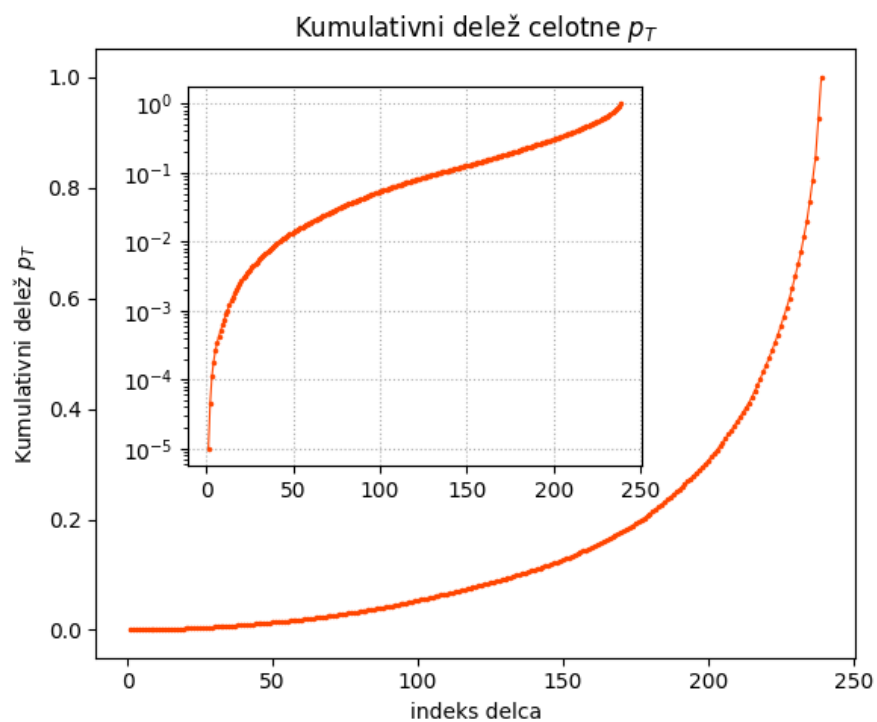
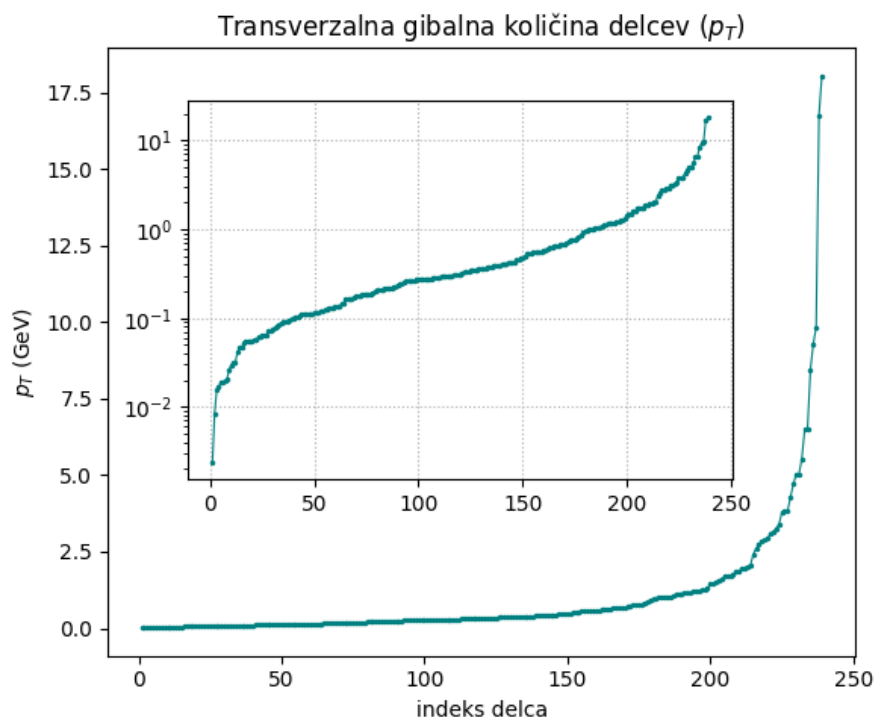
Slika 13: Povprečja in standardne deviacije tisočih izračunov m_H za dano število gruč. Zanimivo je, da pri $K \sim 15$ nekakšno območje minimalne standardne deviacije, ki tudi dokaj dobro napove maso Higgsovega bozona.

Na sliki 12 so prikazani histogrami dobljenih mas za $K = \{10, 20, 30\}$, za 1000 poskusov pri posameznem številu gruč. Opazimo, da porazdelitve vsebujejo mase, ki so daleč od njihovega povprečja. Vidimo, da izmed prikazanih porazdelitev najboljše oceni maso tista z 20 gručami. Ta je tudi najožja. Izgleda, kot da se povprečna izračunana masa manjša z večanjem števila gruč, kar je smiselno saj bomo v gručah imeli posledično manj delcev in s tem tudi manj energije. **Spreminjanje porazdelitve z večanjem števila gruč** pa si lažje predstavljamo z ogledom **animacije** (kjer je ponovno vsak histogram ustvarjen iz 1000 izračunov mas). Na sliki 13 so prikazane izračunane mase ter njihovo odstopanje od eksperimentalne vrednosti, za $K \in [2, 40]$ (graf na levi se začne šele s $K = 5$, zaradi različnih velikostnih redov rešitev). Takoj pade v oči, da ima območje pri $K \sim 15$ najmanjši standardni odklon, ter da v tem območju dokaj dobro ocenimo maso Higgsovega bozona. Z nadaljnjim povečevanjem števila gruč se standardni odklon povečuje, zato lahko to uporabimo kot znak, da že imamo dovolj gruč. V primeru da ne bi poznali eksperimentalno določene mase Higgsovega bozona, bi podrobneje analizirali ravno to območje z najmanjšim standardnim odklonom, saj se napoved mase približno ustali na tistem delu, poleg tega pa očitno še nismo uporabili preveč gruč.

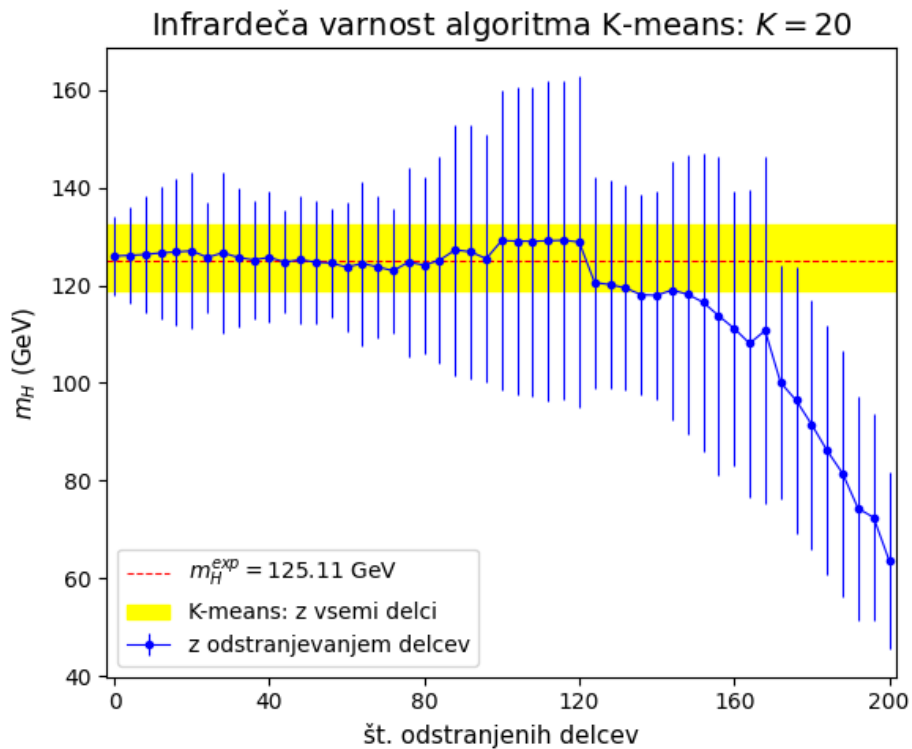
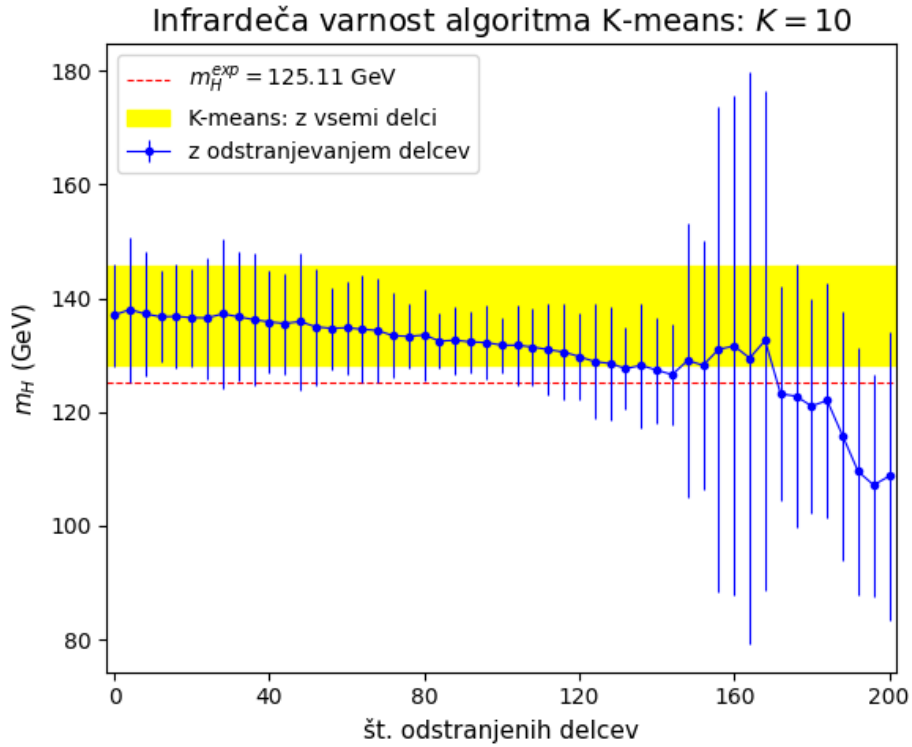
3.4 Testiranje infrardeče varnosti

Izraz infrardeča varnost pomeni, da curki niso senzitivni na dodajanje novih delcev s $p_T \rightarrow 0$. Obratno pomeni tudi, da delci z majhno p_T ne bi smeli bistveno vplivati na rezultate. Zato lahko infrardečo varnost preverimo s postopnim odstranjevanjem delcev z majhnimi p_T . Najprej si pogledjmo, kakšne transverzalne gibalne količine imajo delci iz prvega dogodka iz seznama (slika 14a). Pri postopnem odstranjevanju teh delcev iz podatkov nas zanima tudi, kolikšen del celotne p_T ti izvzeti delci predstavljajo. Kumulativni delež transverzalne gibalne količine do izbranega delca je prikazan na sliki 14b.

Na sliki 15 sta prikazana testa infrardeče varnosti algoritma K-means za 10 in 20 gruč. Pri testih sem postopoma odstranjeval delce z najmanjšo transverzalno gibalno količino p_T , tako da je bilo tekom testa upoštevanih vedno manj delcev, vsak izračun pa sem ponovil 1000-krat in opazoval statistiko le-teh. Za primer 10 gruč vidimo, da se napovedana masa praktično takoj začne manjšati z odstranjevanjem delcev, a sicer ne drastično. V opazovanem dogodku je prisotnih 239 delcev, zato bi lahko predvideli, da se napovedana masa začne dosti hitreje manjšati ko se približujemo primeru, kjer iz računa odstranimo večino delcev. To lahko vidimo tako v primeru 10-ih kot v primeru 20-ih gruč. Slednji se je na testu dosti bolje izkazal. Kljub znatnem padcu napovedane mase po ~ 160 odstranjenih delcih vidimo, da so bili izračuni dosti bolj robustni in je izračunana masa ostala praktično nespremenjena pri izločitvi že znatnega števila delcev.



Slika 14: Transverzalna gibalna količina p_T delcev (zgoraj) in kumulativni delež p_T delcev (spodaj), iz prvega dogodka v seznamu.

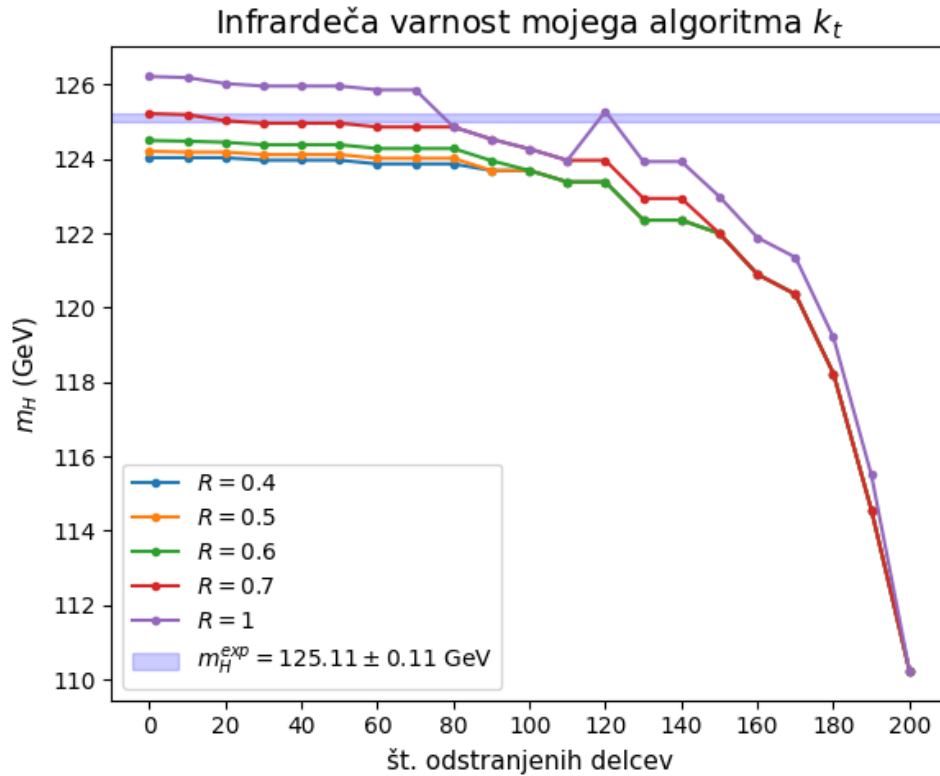


Slika 15: Testa infrardeče varnosti za 10 gruč (zgoraj) in 20 gruč (spodaj). Za vse izračune sem uporabil statistiko 1000, rumeno območje pa predstavlja interval zaupanja ene standardne deviacije izračuna z vsemi delci. Pri $K = 20$ imamo nekakšen plato pri majhnem številu odstranjenih delcev, kar je od algoritma tudi zaželeno!

4 Hierarhično gručenje

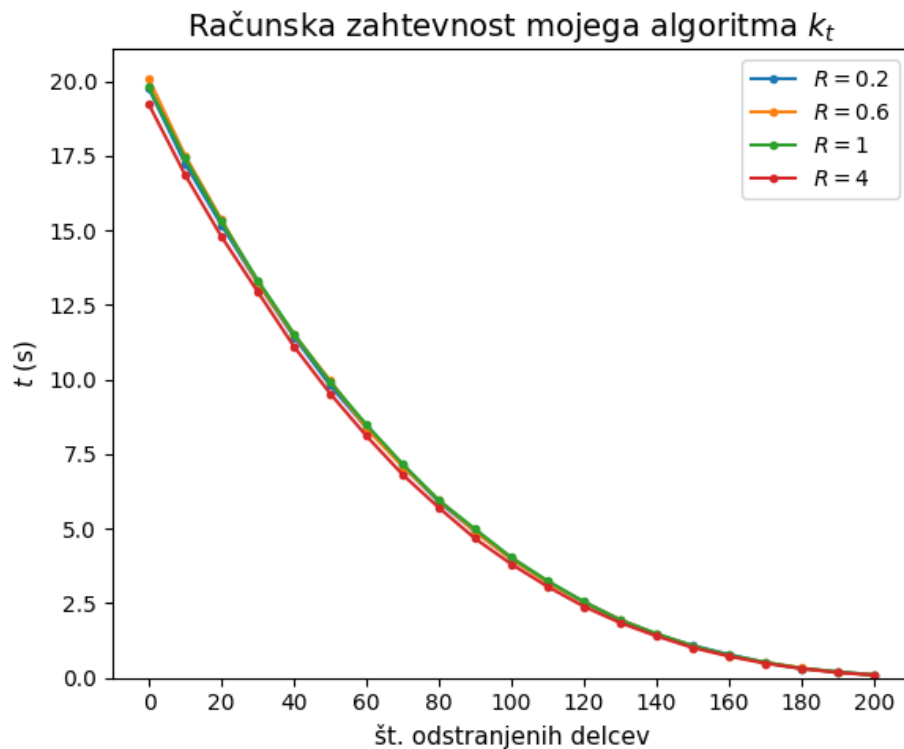
Maso Higgsovega bozona bomo izračunali z uporabo determinističnega algoritma k_t . Najprej bom predstavil rezultate, ki sem jih dobil s svojo implementacijo tega algoritma, nato pa jih primerjal z rezultati, dobljenimi z uporabo knjižnice `pyjet`. Komentar: pri uporabi knjižnice lahko kar neposredno uporabimo podatek za energijo curka, in ni potrebe po uporabi približka $E \approx pc$.

Na sliki 16 so prikazani rezultati moje implementacije algoritma k_t , vključno s testom infrardeče varnosti. Algoritem je test infrardeče varnosti opravil, saj se rezultati ne spremenijo pri odstranjevanju delcev z dovolj majhno p_T . Vidimo, da algoritem z večjim R napove večjo maso, najbližje pravemu rezultatu pa je bil algoritem z $R = 0.7$. Oblika funkcij je približno neodvisna od parametra R , zanimiv pa je skok pri 120 odstranjenih delcih, ki ga doživi krivulja za $R = 1$. Ta ni ravno intuitiven, a izgleda kot da se je morda zmanjšalo število gruč in je bilo nekaj delcev preuvrščenih v gruči z največjo transversalno gibalno količino.



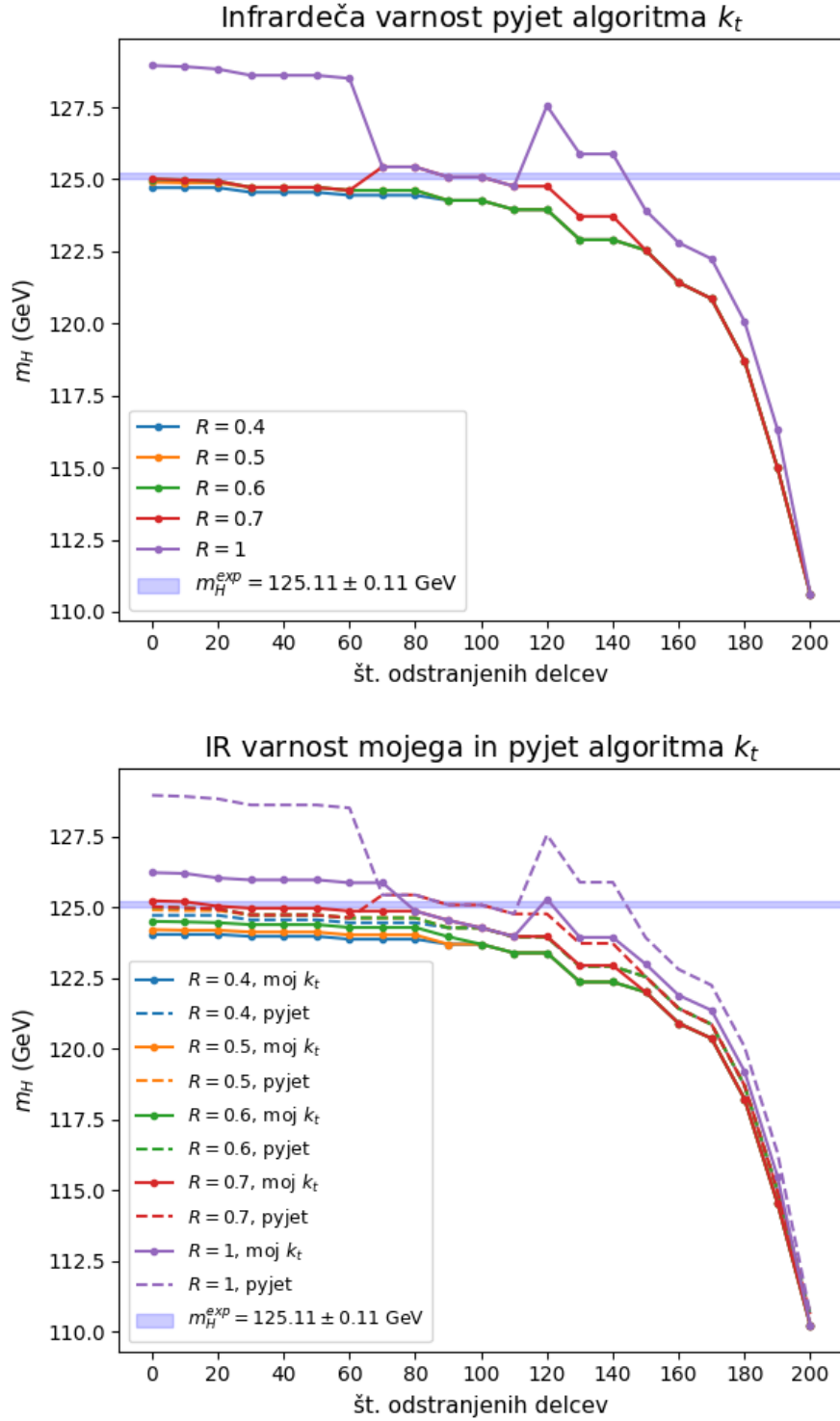
Slika 16: Test infrardeče varnosti moje implementacije algoritma k_t . Plato pri majhnem številu odstranjenih delcev pomeni, da je algoritem test opravil. Poleg tega je algoritem tudi dobro izračunal maso Higgsovega bozona.

Na sliki 17 je prikazana časovna zahtevnost moje implementacije algoritma. Ta ne kaže nobene lepe odvisnosti od parametra R , poleg tega pa je skoraj popolnoma neodvisna od njega.

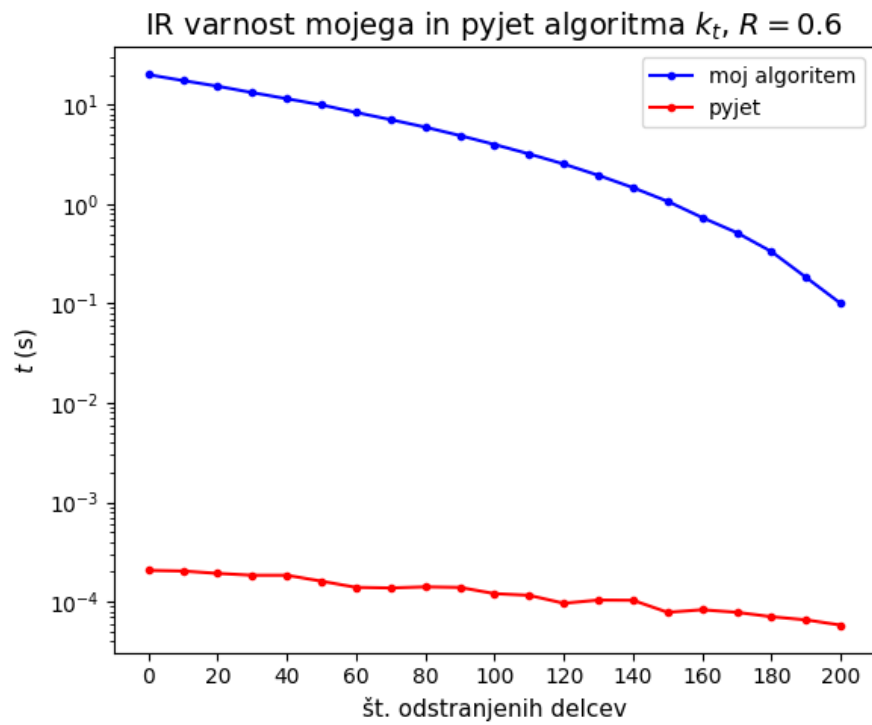
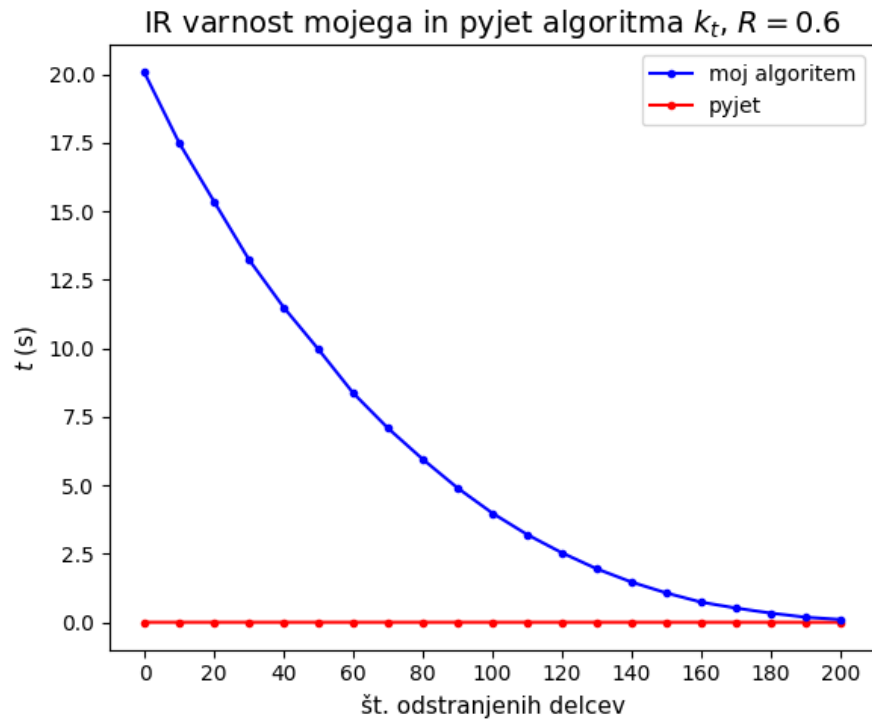


Slika 17: Časovna zahtevnost moje implementacije algoritma k_t . Je skoraj neodvisna od parametra R , ter ne kaže nobenega trenda. Računski časi so bili izračunani s procesorjem Intel-i5 12th Gen.

Na sliki 18 je prikazana primerjava rezultatov `pyjet` algoritma k_t ter moje implementacije. Rezultati so podobni. Časovno zahtevnost obeh algoritmov si lahko ogledamo na sliki 20. Algoritem iz knjižnice `pyjet` je mnogo hitrejši. Za njegovo časovno zahtevnost sem zato tudi vzel povprečje 10 000 izračunov.



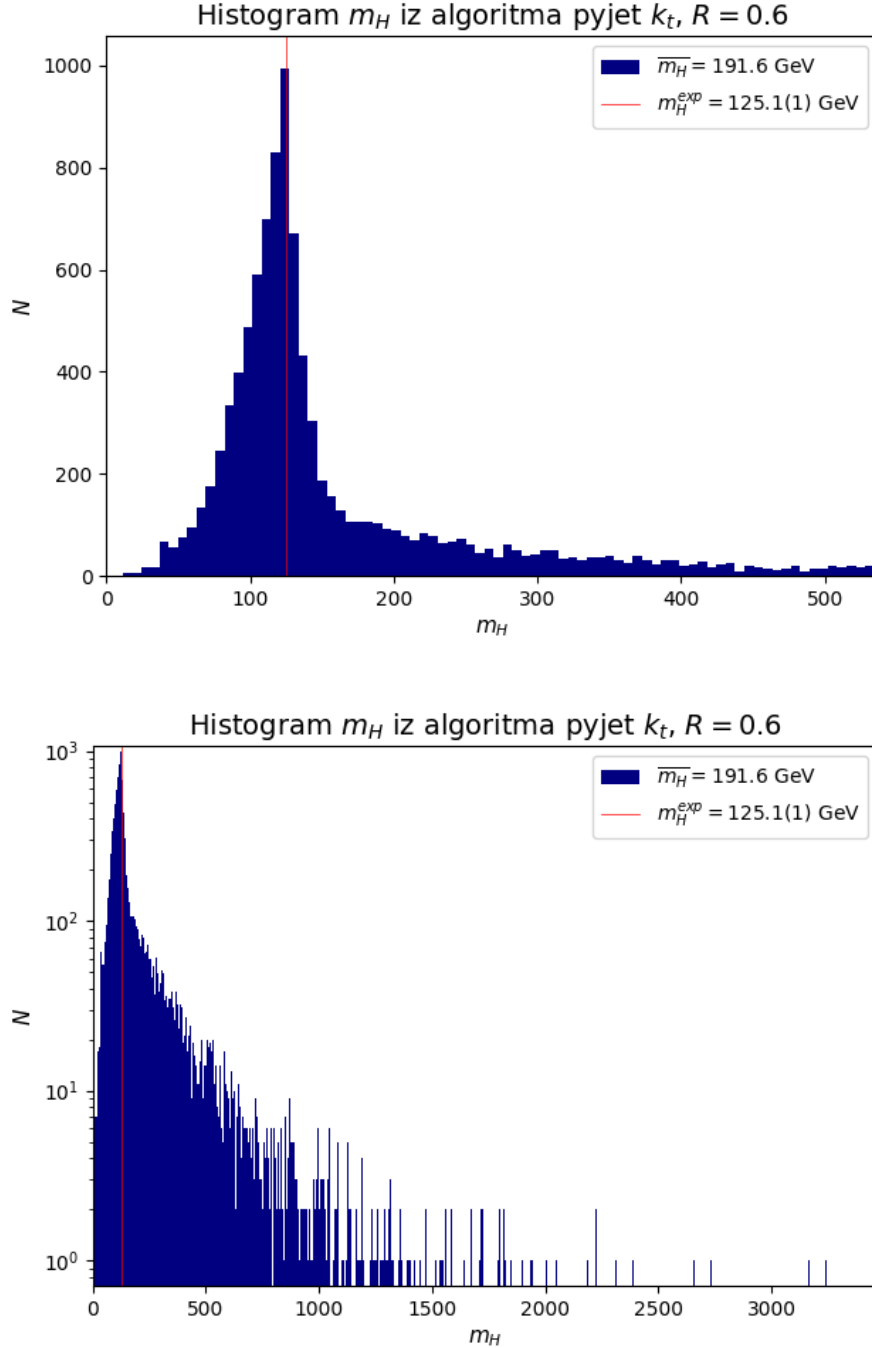
Slika 18: Test infrardeče varnosti na algoritmu pyjet k_t in primerjava rezultatov s tistimi iz moje implementacije. Oblika rešitve zgleda podobno, le vrednosti so nekoliko zama-knjene.



Slika 19: Časovni zahtevnosti mojega in pyjet algoritma k_t . Slednji je mnogo hitrejši. Za njegovo časovno zahtevnost sem vzel povprečje 10 000 izračunov.

4.1 Izračun mase Higgsovega bozona iz vseh dogodkov

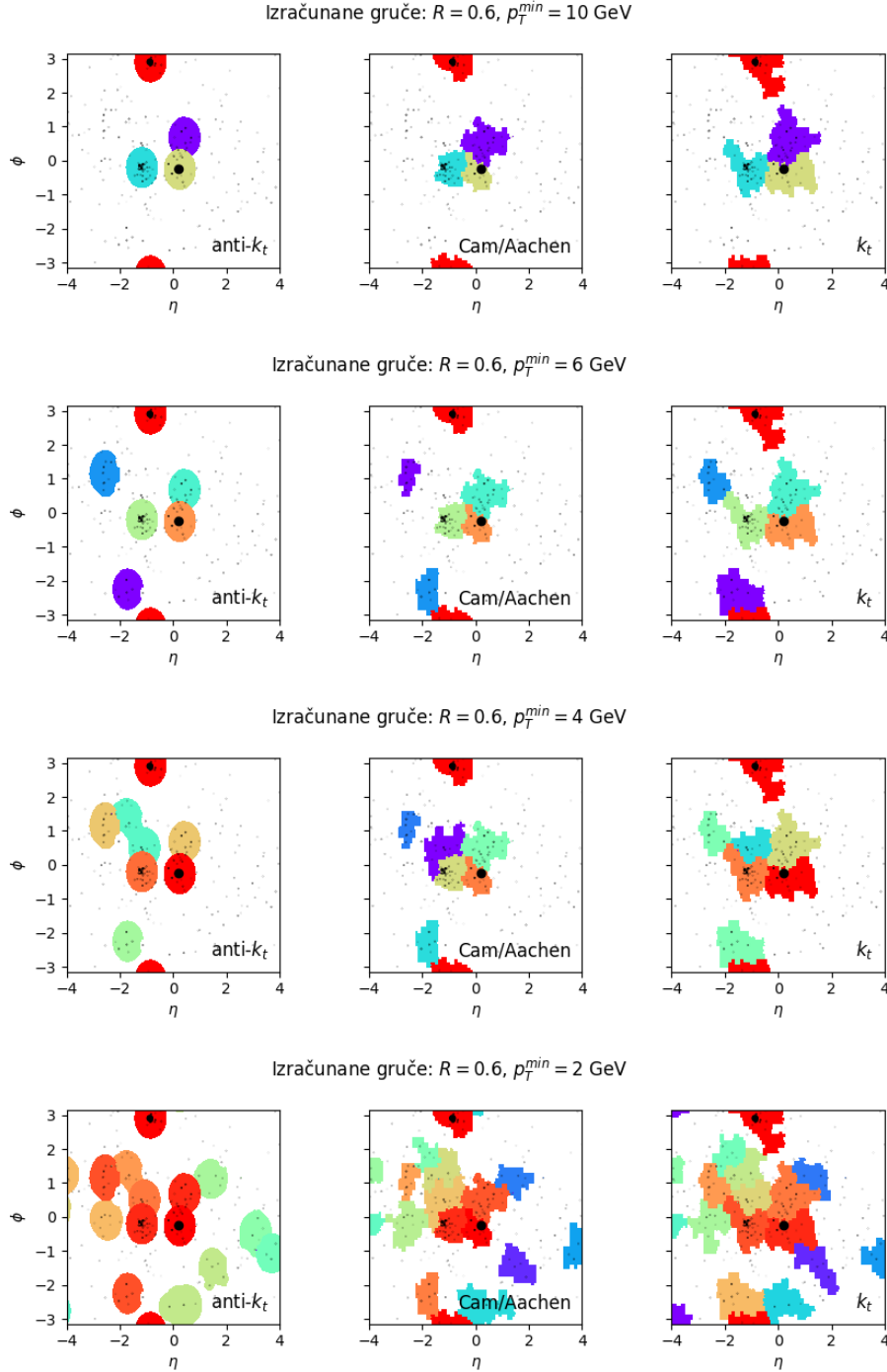
V datoteki imamo na voljo 10 000 dogodkov. Z uporabo algoritma `pyjet` k_t bomo izračunali napovedano maso Higgsovega bozona za vsakega izmed dogodkov, in si nato ogledali statistiko.



Slika 20: Histogram vseh izračunanih mas s k_t iz `pyjet`. Vidimo, da se vrh porazdelitve ujema z maso Higgsovega bozona, medtem ko povprečje maso preceni zaradi anomalij v izračunu.

4.2 Vizualna reprezentacija gruĉ

Oglejmo si še, kako gruĉe izgledajo v primeru hierarhiĉnega gruĉenja. Poleg algoritma k_t bom prikazal tudi rezultate, dobljene z dvema podobnima algoritmoma. Vzel sem fiksni parameter $R = 0.6$, spreminjal pa sem prag minimalne p_T , da je gruĉa prikazana.



Slika 21: Gruĉenja z uporabo algoritmov hierarhiĉnega gruĉenja iz knjiŹnice `pyjet`. Pri-
kazi se razlikujejo le v pragu za transversalno gibalno koliĉino.